



Prueba de desempeño del módulo JavaScript

Esta prueba está diseñada para evaluar tus habilidades en desarrollo web mediante la creación de una aplicación completa que incorpore varias funcionalidades clave.

Objetivo de la prueba

Desarrollar una **Single Page Application (SPA)** utilizando JavaScript (Vanilla JS o Vite), implementando autenticación, manejo de roles, persistencia de sesión y operaciones CRUD consumiendo una API simulada con json-server o externa.

La prueba busca evaluar conocimientos técnicos en:

- JavaScript básico y avanzado
- Manipulación del DOM
- Enrutamiento
- Consumo de APIs
- Persistencia de sesión
- Manejo de roles y autorización
- Arquitectura de aplicaciones SPA
- Buenas prácticas de desarrollo
- Documentación técnica
- Navegación dinámica sin recarga de página

Contexto del problema

Una empresa cuenta con diferentes espacios compartidos para sus colaboradores:

- Oficinas privadas.
- Salas de reuniones.



- Espacios de coworking.
- Auditorios.

Para evitar conflictos de horarios y mejorar la organización, se requiere una plataforma donde los empleados puedan reservar espacios y los administradores puedan gestionar todas las reservas.

Roles del sistema

Administrador (admin)

Puede:

- Ver todas las reservas.
- Crear reservas.
- Editar reservas.
- Eliminar reservas.
- Aprobar o rechazar reservas.
- Gestionar espacios de trabajo (opcional con puntos extra).
- Crear, editar y eliminar espacios (opcional con puntos extra).
- Ver todos los usuarios registrados (opcional con puntos extra).
- Consultar estadísticas generales de uso (opcional con puntos extra).

Usuario (user)

Puede:

- Crear reservas.
- Ver únicamente sus reservas.
- Modificar sus reservas mientras no hayan sido aprobadas.
- Cancelar sus propias reservas.

Restricciones:

- No puede gestionar espacios.
- No puede aprobar o rechazar reservas.
- No puede visualizar reservas de otros usuarios.



- No puede acceder a módulos administrativos.

Requerimientos funcionales

1. Autenticación

La aplicación debe incluir:

- Formulario de inicio de sesión.
- Validación de credenciales contra json-server.
- Manejo de errores de autenticación.
- Persistencia de sesión mediante:
 - localStorage
 - sessionStorage

Debe existir al menos:

- 1 usuario administrador.
- 2 usuarios estándar.

Ejemplo de estructura:



```
{
  "users": [
    {
      "id": 1,
      "name": "Administrador",
      "email": "admin@test.com",
      "password": "Admin123*",
      "role": "admin"
    },
    {
      "id": 2,
      "name": "Usuario",
      "email": "user@test.com",
      "password": "User123*",
      "role": "user"
    }
  ]
}
```



2. Gestión de Reservas

Cada reserva debe contener:

- ID
- Usuario solicitante
- Espacio reservado
- Fecha
- Hora de inicio
- Hora de finalización
- Motivo de la reserva
- Estado:
 - Pendiente



- Aprobada
- Rechazada
- Cancelada

Reglas de negocio

- No se pueden generar reservas duplicadas para el mismo espacio y horario.
- Un usuario solo podrá modificar reservas pendientes.
- Las reservas aprobadas únicamente pueden ser canceladas.
- El administrador puede modificar cualquier reserva.

3. CRUD de Reservas

Implementar:

- GET
- POST
- PUT/PATCH
- DELETE

Permisos

Admin

- CRUD completo sobre cualquier reserva.

User

- Crear reservas.
- Ver únicamente sus reservas.
- Editar reservas pendientes creadas por él.
- Cancelar sus reservas.

4. Gestión de Espacios (opcional con puntos extra)

Cada espacio debe contener:

- ID



- Nombre
- Tipo de espacio
- Capacidad
- Ubicación
- Estado (Disponible / No disponible)

CRUD de espacios (no es obligatorio, será bien valorado).

Solo el administrador podrá:

- Crear espacios.
- Editar espacios.
- Eliminar espacios.
- Consultar todos los espacios.

Si decides crear este módulo, estos espacios deben estar relacionados con las reservas, si no, no tienen validez

5. Protección de rutas

La aplicación debe incluir:

Protección por autenticación

- Usuarios no autenticados no pueden acceder al sistema.

Protección por roles

- Los usuarios con rol user no pueden acceder a:
 - Gestión global de reservas.
 - Módulos administrativos(si decides crearlos).

Comportamiento esperado

- Redirección automática.
- Logout funcional.
- Mensaje de acceso denegado cuando un usuario intenta acceder a una vista restringida.



6. SPA (Single Page Application)

La aplicación debe:

- Navegar sin recargar la página.
- Utilizar renderizado dinámico mediante JavaScript.
- Implementar enrutamiento basado en hash o History API.

7. Persistencia de sesión

Debe implementarse mediante:

- localStorage o sessionStorage.

Requisitos:

- Mantener sesión al refrescar la página.
- Logout funcional.
- Limpieza completa de datos de sesión.

8. Interfaz de usuario

Puede utilizarse:

- CSS
- Bootstrap
- TailwindCSS
- Material UI
- Cualquier librería de estilos

Se evaluará:

- Organización visual.
- Experiencia de usuario.
- Diseño responsive (no obligatorio).
- Claridad de navegación.



9. Requisitos técnicos

Debe incluir:

- Arquitectura modular.
- Manipulación del DOM.
- Manejo de eventos.
- Consumo de APIs mediante Fetch o Axios.
- Manejo de errores.
- Código limpio y reutilizable.
- Protección de rutas mediante middleware o guards simulados.

10. Documentación obligatoria (README.md en inglés)

Debe incluir:

- Project name
- Description
- Technologies used
- Installation
- Running the project
- Running json-server
- Test users
- Project structure
- Role permissions
- Technical decisions

Bonus (Opcional)

- Dashboard con estadísticas.
- Dark mode.
- Buscador de reservas.
- Filtros por fecha.
- Paginación.



- Notificaciones Toast.
- Deploy en Vercel, Netlify o cualquiera de tu preferencia.
- Modulo de control de espacios

Criterio	Porcentaje
Cumplimiento de requerimientos	15%
Comprensión de JavaScript	10%
Enrutamiento SPA	15%
Persistencia y manejo de sesión	10%
Documentación	10%
Sustentación técnica	40%

Nota importante

Para el desarrollo de la prueba se les entregará una base inicial del proyecto que incluirá:

- Proyecto configurado con Vite
- TailwindCSS instalado y funcionando
- Configuración base de json-server
- Estructura mínima de carpetas
- Navegación básica/redireccionamiento inicial

Nota: puede presentar algunos errores al levantar que deberás corregir y sustentar por qué fallaba y cómo se corrige.

La base entregada NO incluirá:

- autenticación
- manejo de sesión
- control de acceso
- protección de rutas
- lógica de roles
- guards



- persistencia de usuario

Un mensaje para ustedes

Esta prueba no marca el final de la aventura en Riwi. Por el contrario, representa un punto de partida para lo que viene en las siguientes etapas de formación y para los retos que encontrarán en escenarios reales de desarrollo.

El objetivo no es únicamente entregar una solución funcional, sino demostrar su capacidad para analizar problemas, tomar decisiones técnicas y construir software de manera organizada. A partir de este momento comienza un proceso de perfilamiento donde identificaremos fortalezas, oportunidades de mejora y áreas en las que cada uno puede seguir creciendo.

La base entregada busca optimizar el tiempo disponible y permitirles enfocarse en la lógica, la arquitectura y la resolución de problemas. Sin embargo, son libres de adaptar, reorganizar o incluso reestructurar completamente el proyecto si consideran que existe una mejor forma de resolverlo.

No se evaluará quién siga exactamente la misma estructura entregada, sino quién logre cumplir de manera efectiva los requerimientos planteados, justificando sus decisiones técnicas y demostrando comprensión de lo que está construyendo.

Recuerden que en el desarrollo de software no siempre existe una única solución correcta. Lo importante es que la solución propuesta sea coherente, mantenible y responda a las necesidades del problema.

Confíen en lo que han aprendido hasta ahora, gestionen bien su tiempo y no tengan miedo de tomar decisiones. Esta prueba es una oportunidad para demostrar todo lo que han construido durante su proceso de formación.

¡Good Luck!



www.riwi.io



301 732 53 27



Cl. 16 # 55 - 129